

Yes. Based on everything we implemented, **Phase 1** should cover only the successful deployment and initial MISP setup before moving to SSL, API integration, and Logstash. Below is the implementation in the exact order we performed it.

Phase 1: MISP Deployment and Initial Configuration

Objective

Deploy a self-hosted MISP (Malware Information Sharing Platform) using Docker Compose on a dedicated Ubuntu VM to serve as the centralized Threat Intelligence Platform for the SOC environment.

Step 1: Prepare Ubuntu Server

Update the operating system.

```
sudo apt update
```

```
sudo apt upgrade -y
```

Step 2: Install Docker

Install Docker Engine.

```
sudo apt install docker.io -y
```

Enable Docker.

```
sudo systemctl enable docker
```

```
sudo systemctl start docker
```

Verify installation.

```
docker --version
```

```
docker ps
```

Step 3: Install Docker Compose

Install Docker Compose plugin.

```
sudo apt install docker-compose-plugin -y
```

Verify installation.

```
docker compose version
```

Step 4: Create Project Directory

Create a dedicated directory for MISP deployment.

```
mkdir ~/misp-docker
```

```
cd ~/misp-docker
```

Step 5: Create Docker Compose Configuration

Create the compose file.

```
nano docker-compose.yml
```

Docker Compose configuration used.

services:

db:

image: mariadb:10.11

container_name: misp-db

restart: unless-stopped

environment:

MYSQL_DATABASE: misp

MYSQL_USER: misp

MYSQL_PASSWORD: misp_password

MYSQL_ROOT_PASSWORD: root_password

volumes:

- mariadb_data:/var/lib/mysql

networks:

- misp-network

redis:

image: redis:7

container_name: misp-redis

restart: unless-stopped

command: redis-server --requirepass redispassword

volumes:

- redis_data:/data

networks:

- misp-network

misp-core:

image: ghcr.io/misp/misp-docker/misp-core:latest

container_name: misp-core

restart: unless-stopped

depends_on:

- db

- redis

environment:

MYSQL_HOST: db

MYSQL_DATABASE: misp

MYSQL_USER: misp

MYSQL_PASSWORD: misp_password

REDIS_FQDN: redis

REDIS_PASSWORD: redispassword

BASE_URL: https://192.168.150.10

ports:

- "80:80"
- "443:443"

volumes:

- misp_files:/var/www/MISP/app/files
- misp_config:/var/www/MISP/app/Config

networks:

- misp-network

volumes:

mariadb_data:

redis_data:

misp_files:

misp_config:

networks:

misp-network:

driver: bridge

Step 6: Deploy Containers

Start the complete MISP stack.

`docker compose up -d`

Verify containers.

`docker ps`

Expected containers.

misp-core

misp-db

misp-redis

Step 7: Verify Container Health

Check running containers.

```
docker ps
```

Inspect logs if required.

```
docker logs misp-core
```

```
docker logs misp-db
```

```
docker logs misp-redis
```

Step 8: Verify Web Interface

Access MISP from a browser.

```
https://<MISP-IP>
```

Initially accessed using the server IP address.

```
https://192.168.150.10
```

Verify the login page loads successfully.

Step 9: Verify Internal Services

Enter the MISP container.

```
docker exec -it misp-core bash
```

Verify Nginx certificate configuration.

```
nginx -T | grep ssl_certificate
```

Expected output.

```
ssl_certificate /etc/nginx/certs/cert.pem;
```

```
ssl_certificate_key /etc/nginx/certs/key.pem;
```

Exit the container.

```
exit
```

Step 10: Verify Persistent Volumes

Confirm that Docker created persistent volumes.

```
docker volume ls
```

Expected volumes.

mariadb_data

redis_data

misp_files

misp_config

These volumes ensure that database contents, uploaded samples, and configuration persist across container restarts.

Step 11: Verify Docker Network

List Docker networks.

`docker network ls`

Inspect the MISP network.

`docker network inspect misp-network`

Verify that all three containers are attached to the same bridge network.

Step 12: Obtain API Authentication Key

Log in to the MISP web interface using the administrator account.

Navigate to:

Administration



List Users



Admin User



Auth Key

Copy the generated API key.

Store it locally.

```
echo "<MISP_API_KEY>" > admin-auth.key
```

Verify.

```
cat admin-auth.key
```

This API key will later be used by the Logstash HTTP Poller for authenticated communication with the MISP REST API.

Step 13: Initial API Verification

Test the REST API using the authentication key.

```
curl -k \
-X POST \
-H "Authorization: <API_KEY>" \
-H "Accept: application/json" \
-H "Content-Type: application/json" \
-d '{
    "returnFormat":"json",
    "limit":1
}' \
https://192.168.150.10/attributes/restSearch
```

A successful response confirms:

- REST API is operational.
- Authentication is valid.
- MISP is serving threat intelligence data.

Components Deployed

Component Image		Purpose
MISP Core	ghcr.io/misp/misp-docker/misp-core:latest	Web interface and Threat Intelligence Platform
MariaDB	mariadb:10.11	Stores events, attributes, users, and metadata
Redis	redis:7	Session management, caching, and background job queue

Deliverables Completed in Phase 1

- Ubuntu server prepared.
- Docker Engine installed.
- Docker Compose installed.
- MISP project directory created.

- Docker Compose deployment completed.
 - MISP Core, MariaDB, and Redis containers deployed.
 - Persistent Docker volumes configured.
 - Docker bridge network configured.
 - MISP web interface verified.
 - Administrator API key generated and stored.
 - REST API connectivity verified using curl.
-

This concludes **Phase 1: MISP Deployment and Initial Configuration**. The next phase would cover **DNS configuration, custom SSL certificate generation, certificate mounting, Java truststore creation, and secure Logstash integration with the MISP REST API**, which is where most of the TLS and integration work began.

Phase 2: DNS Configuration and SSL Certificate Hardening

Objective

Configure a permanent DNS name for the MISP server, replace the default self-signed certificate with a custom certificate containing the correct Subject Alternative Names (SAN), and prepare the environment for secure HTTPS communication with Logstash.

Step 1: Configure Internal DNS

Instead of accessing MISP by IP address, configure a DNS record.

Hostname:

misp.kpk.local

Mapped to:

192.168.150.10

Verify DNS resolution.

host -t a misp.kpk.local

Expected output

misp.kpk.local has address 192.168.150.10

Step 2: Update MISP Base URL

Edit the Docker Compose configuration.

```
cd ~/misp-docker
```

```
nano docker-compose.yml
```


Update

BASE_URL: https://misp.kpk.local

instead of

BASE_URL: https://192.168.150.10

This ensures generated URLs, API endpoints, and notifications consistently use the DNS name.

Step 3: Create a Certificate Configuration File

Create an OpenSSL configuration file.

```
cat > misp-cert.cnf <<EOF
```

```
[req]
```

```
default_bits = 2048
```

```
distinguished_name = req_distinguished_name
```

```
x509_extensions = v3_req
```

```
prompt = no
```

```
[req_distinguished_name]
```

```
CN = misp.kpk.local
```

```
[v3_req]
```

```
subjectAltName = @alt_names
```

```
[alt_names]
```

```
DNS.1 = misp.kpk.local
```

```
DNS.2 = localhost
```

```
IP.1 = 192.168.150.10
```

```
IP.2 = 127.0.0.1
```

```
EOF
```

Purpose

- Define the Common Name (CN)
- Configure Subject Alternative Names (SAN)
- Generate a certificate trusted for both DNS and IP access

Step 4: Generate a New Self-Signed Certificate

Remove any previous certificates.

```
rm -f cert.pem key.pem
```

Generate the certificate.

```
openssl req \  
-x509 \  
-nodes \  
-days 365 \  
-newkey rsa:2048 \  
-keyout key.pem \  
-out cert.pem \  
-config misp-cert.cnf
```

Generated files

cert.pem

key.pem

Step 5: Verify Certificate Contents

Inspect the certificate.

```
openssl x509 \  
-in cert.pem \  
-text \  
-noout | grep -A2 "Subject Alternative"
```

Expected output

DNS:misp.kpk.local

DNS:localhost

IP Address:192.168.150.10

IP Address:127.0.0.1

This confirms the certificate contains the required Subject Alternative Names.

Step 6: Create a Directory for Certificates

Create a dedicated directory.

```
mkdir certs
```

Move certificates.

```
mv cert.pem certs/
```

```
mv key.pem certs/
```

Project structure

```
misp-docker/
```

```
|
```

```
| └─ certs/
```

```
|   └─ cert.pem
```

```
|   └─ key.pem
```

```
|
```

```
| └─ docker-compose.yml
```

```
| └─ admin-auth.key
```

```
└─ misp-cert.cnf
```

Using a dedicated directory simplifies certificate replacement in the future.

Step 7: Mount Certificates into the Container

Edit Docker Compose.

```
nano docker-compose.yml
```

Add bind mounts.

volumes:

- misp_files:/var/www/MISP/app/files
- misp_config:/var/www/MISP/app/Config

- ./certs/cert.pem:/etc/nginx/certs/cert.pem:ro
- ./certs/key.pem:/etc/nginx/certs/key.pem:ro

Benefits

- Certificates remain outside the container.
- Future certificate updates require only replacing host files.
- Certificates persist across container recreation.

Step 8: Recreate the Containers

Stop the existing stack.

```
docker compose down
```

Start the updated stack.

```
docker compose up -d
```

Verify containers.

```
docker ps
```

Expected containers

```
misp-core
```

```
misp-db
```

```
misp-redis
```

Step 9: Verify Mounted Certificates

Access the MISP container.

```
docker exec -it misp-core bash
```

Verify mounted files.

```
ls -l /etc/nginx/certs/
```

Inspect the active certificate.

```
openssl x509 \
```

```
-in /etc/nginx/certs/cert.pem \
```

```
-text \
```

```
-noout | grep -A2 "Subject Alternative"
```

Confirm the container is using the newly mounted certificate.

Exit.

```
exit
```

Step 10: Verify HTTPS Endpoint

Test HTTPS access.

```
curl -k https://misp.kpk.local
```

or simply access

`https://misp.kpk.local`

from a browser.

Successful access confirms

- HTTPS is functioning.
 - Nginx is using the mounted certificate.
 - DNS resolution is correct.
-

Step 11: Verify REST API over HTTPS

Test the API.

```
curl -k \  
-X POST \  
-H "Authorization: <API_KEY>" \  
-H "Accept: application/json" \  
-H "Content-Type: application/json" \  
-d '{  
  "returnFormat":"json",  
  "type":["ip-dst"],  
  "limit":1  
}' \  
  
https://misp.kpk.local/attributes/restSearch
```

A successful JSON response confirms:

- DNS configuration is operational.
 - HTTPS communication works correctly.
 - The certificate is accepted by clients using -k.
 - The MISP REST API is accessible via its permanent hostname.
-

Deliverables Completed in Phase 2

- Internal DNS record (`misp.kpk.local`) configured.
- `BASE_URL` updated to use the DNS hostname.
- Custom OpenSSL configuration created.
- Self-signed TLS certificate generated with appropriate SAN entries.

- Dedicated certs/ directory created.
 - Certificate and private key mounted into the MISP container.
 - MISP containers recreated to apply the new configuration.
 - Active certificate verified inside the container.
 - HTTPS access validated using the DNS hostname.
 - REST API connectivity confirmed over HTTPS.
-

At the end of **Phase 2**, MISP was securely accessible using `https://misp.kpk.local` with a custom certificate containing the correct Subject Alternative Names. This completed the TLS hardening required before integrating Logstash, which is covered in **Phase 3: Java Truststore Creation and Logstash–MISP Integration**.

Phase 3: Java Truststore Configuration and Logstash–MISP Integration

Objective

Configure Logstash to securely communicate with the MISP REST API over HTTPS by creating a Java Truststore containing the MISP certificate, configuring Logstash to trust the certificate, and successfully ingest MISP threat intelligence into Elasticsearch.

Step 1: Copy the MISP Certificate to the SOC Server

Copy the certificate generated on the MISP server.

```
scp cert.pem soc-core@<SOC-IP>:~/project/certs/
```

Verify.

```
cd ~/project/certs
```

```
ls
```

Expected

```
cert.pem
```

Step 2: Install Java Runtime

The Java keytool utility is required to create a truststore.

```
sudo apt update
```

```
sudo apt install openjdk-17-jre-headless -y
```

Verify.

```
keytool -help
```

Step 3: Create a Java Truststore

Import the certificate.

```
keytool -importcert \  
-alias misp-kpk \  
-file cert.pem \  
-keystore truststore.jks
```

Choose a password for the truststore.

Example

changeit

Confirm

Trust this certificate?

yes

Step 4: Verify the Truststore

List the imported certificates.

```
keytool -list \  
-keystore truststore.jks
```

Expected

Your keystore contains:

misp-kpk

Inspect the certificate.

```
keytool \  
-list \  
-v \  
-alias misp-kpk \  
-keystore truststore.jks
```

Verified information

- Owner

CN=misp.kpk.local

- Subject Alternative Names

DNS:misp.kpk.local

DNS:localhost

IP:192.168.150.10

IP:127.0.0.1

This confirmed the truststore contained the updated certificate.

Step 5: Configure Logstash to Use the Truststore

Edit the Logstash Docker Compose configuration.

environment:

LS_JAVA_OPTS: >

-Djavax.net.ssl.trustStore=/usr/share/logstash/config/certs/truststore.jks

-Djavax.net.ssl.trustStorePassword=<PASSWORD>

Mount the certificate directory.

volumes:

- ./certs:/usr/share/logstash/config/certs:ro

This allows the Java HTTP client inside Logstash to trust the MISP HTTPS certificate.

Step 6: Configure the HTTP Poller Pipeline

Create the MISP IOC pipeline.

05-misp-ioc.conf

Main input configuration

```
http_poller {
```

```
  urls => {
```

```
    misp_iocs => {
```

```
      method => post
```

```
      url => "https://misp.kpk.local/attributes/restSearch"
```



```

headers => {
  Authorization => "<API_KEY>"
  Accept => "application/json"
  "Content-Type" => "application/json"
}

body => '{
  "returnFormat":"json",
  "type":["ip-dst","domain","url"],
  "limit":100
}'
}
}

```

```

schedule => { cron => "*" * " * * * * " }

```

```

codec => json
}

```

Polling interval

- Every minute

Data requested

- Domains
- URLs
- Destination IPs

Step 7: Parse Returned Attributes

Split the returned array.

```

split {
  field => "[response][Attribute]"
}

```

Normalize the fields.

```
rename => {  
  "[response][Attribute][type]" => "[threat][indicator][type]"  
}
```

```
rename => {  
  "[response][Attribute][value]" => "[threat][indicator][value]"  
}
```

```
rename => {  
  "[response][Attribute][uuid]" => "[threat][indicator][reference]"  
}
```

Add ECS-compatible metadata.

```
add_field => {  
  "[event][dataset]" => "misp.ioc"  
  "[event][kind]" => "enrichment"  
  "[event][category]" => "threat"  
  "[threat][indicator][provider]" => "MISP"  
}
```

Normalize IOC values.

```
lowercase => [  
  "[threat][indicator][type]",  
  "[threat][indicator][value]"  
]
```

Step 8: Output to Elasticsearch

Configure the Elasticsearch output.

```
elasticsearch {  
  
  hosts => ["http://elasticsearch:9200"]  
  
  index => "misp-ioc-intelligence"
```

```
user => "elastic"
```

```
password => "${ELASTIC_PASSWORD}"
```

```
}
```

A debug output was also enabled.

```
stdout {
```

```
  codec => rubydebug
```

```
}
```

Step 9: Restart Logstash

Apply the updated configuration.

```
docker restart logstash
```

Step 10: Initial SSL Troubleshooting

The first pipeline execution failed with

Certificate for <misp.kpk.local> doesn't match any of the subject alternative names

Root cause

- The default MISP certificate only contained

localhost

127.0.0.1

Resolution

- Generated a new certificate with the required Subject Alternative Names.
- Mounted the new certificate into the MISP container.
- Created a new Java truststore.
- Imported the updated certificate.
- Restarted Logstash.

After these changes, HTTPS communication succeeded.

Step 11: Validate the MISP REST API

Test directly.

```
curl \
-X POST \
-H "Authorization: <API_KEY>" \
-H "Accept: application/json" \
-H "Content-Type: application/json" \
-d '{
  "returnFormat": "json",
  "type": ["ip-dst"],
  "limit": 1
}' \
https://misp.kpk.local/attributes/restSearch
```

A JSON response confirmed

- HTTPS connectivity
- API authentication
- REST endpoint functionality

Step 12: Verify Elasticsearch Ingestion

Check the number of imported IOC documents.

```
curl -u elastic:<PASSWORD> \
http://localhost:9200/misp-ioc-intelligence/_count?pretty
```

The document count increased continuously as Logstash polled MISP.

Inspect the data.

```
curl -u elastic:<PASSWORD> \
-X GET \
"http://localhost:9200/misp-ioc-intelligence/_search?pretty"
```

Verified fields included

threat.indicator.type

threat.indicator.value

threat.indicator.reference

threat.indicator.provider

event.dataset

event.kind

event.category

The pipeline successfully normalized MISP IOC attributes into an ECS-aligned format.

Step 13: Confirm Continuous Synchronization

The HTTP Poller executed automatically every minute.

Workflow

MISP

|



REST API (/attributes/restSearch)

|



Logstash HTTP Poller

|



Filter & Normalize

|



Elasticsearch

|



IOC Intelligence Index

This provided automatic synchronization of threat indicators from MISP into Elasticsearch.

Deliverables Completed in Phase 3

- Installed Java runtime and keytool.
- Copied the MISP TLS certificate to the SOC server.
- Created a Java Truststore (truststore.jks) containing the MISP certificate.
- Configured Logstash to use the Truststore for HTTPS validation.
- Implemented a Logstash http_poller pipeline to query the MISP REST API every minute.

- Parsed, normalized, and enriched MISP IOC data using Logstash filters.
- Indexed threat indicators into the misp-ioc-intelligence Elasticsearch index.
- Resolved SSL certificate validation issues by regenerating the MISP certificate with correct Subject Alternative Names and updating the Truststore.
- Verified secure API communication, successful IOC ingestion, and continuous synchronization from MISP to Elasticsearch.

At the end of **Phase 3**, the threat intelligence pipeline was fully operational: MISP securely exposed threat indicators over HTTPS, Logstash periodically retrieved and normalized them, and Elasticsearch stored them in a dedicated intelligence index ready for visualization and detection rule development.

Phase 4: MISP Threat Intelligence Feed Integration and Detection Rule Development

Objective

Integrate complete MISP event data into Elasticsearch, normalize threat intelligence documents, develop Elastic Detection Rules based on MISP threat feeds, and validate the generated alerts.

Step 1: Create a Dedicated MISP Threat Feed Pipeline

A separate Logstash pipeline was created to ingest complete MISP event data.

Example pipeline file

06-misp-threat.conf

The pipeline periodically queried the MISP REST API to retrieve published threat events.

Step 2: Configure the HTTP Poller

The pipeline used the MISP REST API.

```
input {
```

```
  http_poller {
```

```
    urls {
```

```
      misp_events {
```

```
        method => post
```

```
url => "https://misp.kpk.local/events/restSearch"
```

```
headers {
```

```
    Authorization => "<API_KEY>"
```

```
    Accept => "application/json"
```

```
    "Content-Type" => "application/json"
```

```
}
```

```
body => '{
```

```
    "returnFormat":"json"
```

```
}'
```

```
}
```

```
}
```

```
schedule => { cron => "*" * * * * }
```

```
codec => json
```

```
}
```

```
}
```

The HTTP Poller automatically synchronized threat events every minute.

Step 3: Parse MISP Events

The returned MISP response contains nested event objects.

The Logstash filters extracted useful metadata including

- Event ID

- Event UUID
- Event Information
- Threat Level
- Analysis Status
- Distribution
- Publish Timestamp
- Source Organization
- Event Date

These fields were normalized before indexing.

Step 4: Normalize Threat Metadata

Important fields indexed included

info

source

threat_level_id

publish_timestamp

distribution

analysis

event_id

uuid

These fields became searchable inside Elasticsearch.

Step 5: Create the Threat Intelligence Index

Threat events were stored in a dedicated index.

misp-threat-intelligence

This index remained separate from IOC indicators.

Architecture

MISP

|

└───▶ IOC Pipeline

| |

| ▼

| misp-ioc-intelligence

|

▼

Threat Feed Pipeline

|

▼

misp-threat-intelligence

Step 6: Validate Data Ingestion

Verify the document count.

```
curl -u elastic:<PASSWORD> \
http://localhost:9200/misp-threat-intelligence/_count?pretty
```

Verify stored documents.

```
curl -u elastic:<PASSWORD> \
-X GET \
"http://localhost:9200/misp-threat-intelligence/_search?pretty"
```

Verified fields included

info

source

publish_timestamp

threat_level_id

@timestamp

Step 7: Inspect Index Mapping

Mappings were examined before writing detection rules.

Example

```
curl -u elastic:<PASSWORD> \
```

```
http://localhost:9200/misp-threat-intelligence/_mapping?pretty
```

Important observations

- @timestamp mapped as a date.
- publish_timestamp stored as text.
- threat_level_id stored as text.

These mappings influenced the rule queries.

Step 8: Develop Detection Rules

Multiple Elastic Detection Rules were created based on the MISP threat feed.

Rule 1: Published Critical MISP Threat

Purpose

Detect newly ingested MISP events with the highest threat severity.

Primary fields

source = MISP

threat_level_id = 1

Severity

Critical

Risk Score

99

Rule 2: Critical MISP Threat Intelligence

Purpose

Alert on all critical MISP intelligence events.

Primary conditions

source = MISP

threat_level_id = 1

Severity

High

Risk Score

90

Rule 3: New MISP Threat Intelligence

Purpose

Identify newly synchronized MISP events.

Primary fields

source = MISP

Time filter

@timestamp

Severity

Medium

Risk Score

49

The rule used the Elasticsearch ingestion timestamp rather than publish_timestamp, because publish_timestamp was indexed as text and was not suitable for time-range queries.

Step 9: Rule Testing

Several Elasticsearch queries were used to verify the indexed data before enabling the rules.

Example

Search by threat level.

```
{  
  "query": {  
    "term": {  
      "threat_level_id.keyword": "1"
```

```
}  
}  
}
```

Search for source.

```
{  
  "query": {  
    "term": {  
      "source.keyword": "MISP"  
    }  
  }  
}
```

Inspect recent documents.

```
{  
  "sort": {  
    "@timestamp": "desc"  
  }  
}
```

These validation queries ensured that the detection logic matched the indexed data.

Step 10: Rule Validation in Kibana

After enabling the rules, Kibana Detection Engine generated alerts for

- New MISP threat events
- Critical threat intelligence
- Published critical threat intelligence

Rule execution status was monitored from the Detection Rules dashboard to verify successful execution.

Step 11: IOC Correlation Rule Evaluation

A correlation rule was also evaluated to match network events against IOC values stored in the misp-ioc-intelligence index.

The intended workflow was:

Network Logs

|
▼
Destination IP / Domain / URL

|
▼
Elastic Detection Rule

|
▼
IOC Match

|
▼
Critical Alert

During validation, it was determined that reliable end-to-end testing required properly generated Suricata network events. Since Suricata validation was outside the current implementation scope, full functional testing of the IOC correlation rule was deferred while the ingestion pipeline itself was confirmed to be operating correctly.

Step 12: End-to-End Threat Intelligence Workflow

Final workflow

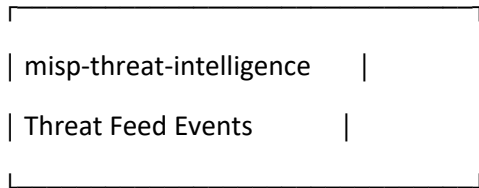
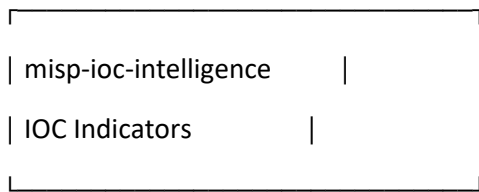
MISP Threat Events

|
▼
MISP REST API

|
▼
Logstash HTTP Poller

|
▼
Normalization

|
▼
Elasticsearch



Elastic Detection Rules



Security Alerts Dashboard

Deliverables Completed in Phase 4

- Implemented a dedicated Logstash pipeline for ingesting complete MISP threat events.
- Parsed and normalized MISP event metadata into the misp-threat-intelligence Elasticsearch index.
- Validated indexed documents and reviewed field mappings to support rule development.
- Developed three MISP-based Elastic Detection Rules:
 - **Published Critical MISP Threat**
 - **Critical MISP Threat Intelligence**
 - **New MISP Threat Intelligence**
- Validated rule execution using Elasticsearch queries and the Kibana Detection Engine.

- Evaluated an IOC correlation rule design for matching network telemetry with MISP indicators, with full validation deferred pending Suricata-generated traffic.
- Established an automated threat intelligence workflow from MISP ingestion through Elasticsearch indexing to Elastic Security alert generation.